

Guia do Aluno

Arquivos com `__`duplo underscore`__`

`__init__.py` `__main__.py` `__repr__` `__str__` e outros

O que são, para que servem e quando você vai encontrá-los

2026

1. O que é o "dunder underscore"?

Em Python, o underscore é o caractere `_` (underline, traço embaixo). Quando aparece dois de cada lado de um nome, `__` assim `__` ele tem um significado especial na linguagem.

A comunidade Python chama esse padrão de "dunder" (contração de double underscore). Então `__init__` se lê "dunder init", `__repr__` se lê "dunder repr", e por aí vai.

O nome "dunder"

"Dunder" = double underscore = duplo underscore

`__init__` → "dunder init"
`__repr__` → "dunder repr"
`__str__` → "dunder str"
`__main__` → "dunder main"

São nomes reservados pelo Python -- têm comportamento especial definido pela linguagem.

O duplo underscore aparece em dois contextos diferentes que vamos ver neste guia:

- Arquivos: `__init__.py`, `__main__.py` — organização de projetos
- Métodos de classes: `__init__`, `__repr__`, `__str__` — comportamento especial de objetos

Nomes reservados

Nunca crie variáveis, funções ou arquivos com duplo underscore para uso próprio.

Esses nomes são reservados pelo Python. Usar fora do contexto certo vai gerar confusão

e pode sobrescrever comportamentos internos da linguagem.

2. O arquivo `__init__.py`

2.1 O problema que ele resolve

Imagine que você tem um projeto com esta estrutura de pastas:

```
meu_projeto/  
├── app/  
│   ├── models.py  
│   ├── database.py  
│   └── routers/  
│       └── usuarios.py  
└── main.py
```

O `main.py` quer importar o `models.py` que está dentro da pasta `app/`. Sem o `__init__.py`, o Python não sabe que a pasta `app/` é um "pacote" — ela é só uma pasta comum do sistema operacional.

```
# main.py -- tentativa de importar sem __init__.py  
from app.models import Funcionario  
  
# Resultado:  
# ModuleNotFoundError: No module named "app"
```

A solução: criar um arquivo `__init__.py` dentro de cada pasta que você quer usar como pacote Python.

```
meu_projeto/  
├── app/  
│   ├── __init__.py    ← torna app/ um pacote Python  
│   ├── models.py  
│   ├── database.py  
│   └── routers/  
│       ├── __init__.py ← torna routers/ um pacote Python  
│       └── usuarios.py  
└── main.py
```

```
# main.py -- agora funciona  
from app.models import Funcionario    # ☒ funciona  
from app.routers.usuarios import router # ☒ funciona
```

2.2 O que vai dentro do `__init__.py`?

Na maioria dos casos: nada. O arquivo pode ficar completamente vazio. A simples presença dele já é suficiente para transformar a pasta em um pacote Python.

Arquivo vazio é suficiente

`__init__.py` vazio é a forma mais comum e mais simples.

Não precisa escrever nada dentro — só criar o arquivo.

No VS Code: botão direito na pasta → New File → `__init__.py`

Porém, você pode usar o `__init__.py` para facilitar imports. Veja o exemplo:

```
# Sem nada no __init__.py:
from app.models import Funcionario, Departamento, Cargo

# Com o __init__.py configurado assim:
# app/__init__.py
from app.models import Funcionario, Departamento, Cargo

# Agora o import fica mais curto:
from app import Funcionario, Departamento, Cargo
```

No nosso curso

Nos projetos do curso (Módulos 2 e 3) os `__init__.py` ficam vazios.

A versão com imports no `__init__.py` é usada em bibliotecas maiores.

Por enquanto: criar vazio e não se preocupar com o conteúdo.

2.3 Analogia para fixar

Pense em um prédio de escritórios:

- A pasta (diretório) é como um andar do prédio
- Os arquivos `.py` são as salas com funcionários dentro
- O `__init__.py` é a recepção do andar — sem ela, ninguém sabe que o andar existe e não consegue acessar as salas

Mundo real	Python
Prédio completo	Projeto Python
Andar sem recepção	Pasta sem <code>__init__.py</code> (não é pacote)
Andar com recepção	Pasta com <code>__init__.py</code> (é um pacote)
Sala do andar	Arquivo <code>.py</code> dentro da pasta
Acessar uma sala	Fazer import de um módulo

2.4 Perguntas frequentes sobre `__init__.py`

Preciso criar em toda pasta do projeto?

Não — só nas pastas que você vai usar em imports. O exemplo abaixo mostra quais precisam:

```
meu_projeto/
├── app/
│   ├── __init__.py    ← PRECISA  (importamos de app.models,
│   │                   app.database)
│   ├── models.py
│   ├── database.py
│   └── routers/
│       ├── __init__.py ← PRECISA  (importamos de
│       │                   app.routers.usuarios)
│       └── usuarios.py
├── venv/               ← NÃO PRECISA (nunca importamos de venv)
├── main.py             ← arquivo normal, não é pasta
└── requirements.txt    ← arquivo normal, não é pasta
```

O que acontece se eu esquecer de criar?

```
# Erro típico quando falta o __init__.py:
ModuleNotFoundError: No module named "app"

# Ou:
ImportError: cannot import name "Funcionario" from "app"

# Solução: criar o arquivo __init__.py vazio na pasta app/
```

Por que o VS Code às vezes não reclama mas o terminal reclama?

O VS Code tem seu próprio sistema de análise de código que às vezes ignora esse erro. Mas quando você executa `python main.py` no terminal, o Python real verifica e dá o erro. Sempre confiar no terminal, não só no VS Code.

3. O arquivo `__main__.py`

O `__main__.py` permite executar uma pasta inteira como se fosse um único script Python. Você usa o comando `python -m nome_da_pasta` em vez de `python nome_da_pasta/main.py`.

3.1 Como funciona

```
# Sem __main__.py:
python app/main.py           # precisa apontar para o arquivo

# Com __main__.py dentro de app/:
python -m app                # executa app/__main__.py automaticamente

# Ou executar um módulo diretamente:
python -m app.seed           # executa app/seed.py como script
```

Você já usou isso no curso! Quando rodamos:

```
# Aula 14 -- rodar o seed como módulo
python -m app.seed

# O -m diz ao Python: execute o arquivo app/seed.py como script
# É diferente de: python app/seed.py
# A diferença: com -m, os imports relativos funcionam corretamente
```

3.2 Conteúdo típico do `__main__.py`

```
# app/__main__.py
from app.database import engine, Base
from app import models
from app.seed import popular_banco

if __name__ == "__main__":
    Base.metadata.create_all(bind=engine)
    popular_banco()
    print("Aplicação iniciada!")
```

Quando você vai usar

O `__main__.py` é opcional -- a maioria dos projetos do curso não precisa dele. Você vai encontrá-lo em bibliotecas que podem ser executadas diretamente. Exemplo: `python -m pytest` (o `pytest` tem um `__main__.py` interno)

4. O bloco `if __name__ == "__main__"`

Esse é o padrão mais importante de todos para entender — e aparece em praticamente todo arquivo Python profissional.

4.1 O problema

Imagine que você tem dois arquivos:

```
# calculos.py
def somar(a, b):
    return a + b

# Testando a função:
resultado = somar(2, 3)
print(f"Resultado: {resultado}") # imprime 5

# main.py
from calculos import somar # ← importa a função

# Problema: ao importar calculos.py, o Python EXECUTA todo o arquivo
# Então o print("Resultado: 5") vai aparecer no terminal
# mesmo que você não queira!

# Saída indesejada:
# Resultado: 5          ← veio do calculos.py ao ser importado
# (só queria usar a função somar, não ver esse print)
```

4.2 A solução: `__name__`

A variável `__name__` é definida automaticamente pelo Python em todo arquivo. O valor dela depende de como o arquivo está sendo usado:

Situação	Valor de <code>__name__</code>
Você executa o arquivo diretamente: <code>python calculos.py</code>	<code>"__main__"</code>
O arquivo é importado por outro: <code>from calculos import somar</code>	<code>"calculos"</code> (nome do arquivo)

Com isso, podemos separar o que roda sempre do que roda só quando o arquivo é executado diretamente:

```
# calculos.py -- versão correta
def somar(a, b):
    return a + b

# Este bloco SÓ executa quando rodamos: python calculos.py
# Quando alguém importa calculos.py, este bloco É IGNORADO
```

```
if __name__ == "__main__":
    resultado = somar(2, 3)
    print(f"Resultado: {resultado}") # só aparece se rodar direto
```

```
# main.py -- agora funciona corretamente
from calculos import somar

# O print do calculos.py NÃO aparece mais
# Porque ao importar, __name__ == "calculos" (não "__main__")

print(somar(10, 5)) # 15
```

Resumo do padrão

O `if __name__ == "__main__"` significa:

"Execute este bloco SOMENTE se este arquivo for o ponto de entrada do programa"

Use sempre que quiser testar ou demonstrar algo no arquivo,
mas sem que o código rode quando o arquivo for importado.

4.3 Onde você já viu isso no curso

```
# app/seed.py -- Módulo 3
def popular_banco():
    # ... código de seed ...
    pass

if __name__ == "__main__":
    popular_banco()

# Resultado:
# python -m app.seed      → popular_banco() é chamada (ponto de
#                          entrada)
# from app.seed import ... → popular_banco() NÃO é chamada
#                          automaticamente
```

5. Métodos dunder em classes

Além dos arquivos, o duplo underscore aparece como métodos especiais dentro de classes. Eles definem comportamentos que o Python aciona automaticamente em certas situações.

5.1 `__init__` — o construtor

O `__init__` é executado automaticamente toda vez que você cria um objeto da classe. É onde você define os atributos iniciais do objeto.

```
class Funcionario:
    def __init__(self, nome, salario):
        # Este bloco executa quando fazemos: Funcionario("Carlos",
        4500)
        self.nome = nome
        self.salario = salario
        self.ativo = True # valor padrão

# Criar um objeto chama __init__ automaticamente:
f = Funcionario("Carlos", 4500)
# Python faz internamente: Funcionario.__init__(f, "Carlos", 4500)

print(f.nome) # Carlos
print(f.salario) # 4500
print(f.ativo) # True
```

O parâmetro self

`self` é sempre o primeiro parâmetro do `__init__` e de todo método de instância. `self` representa o próprio objeto que está sendo criado ou manipulado. Você nunca passa `self` explicitamente -- o Python passa automaticamente.

5.2 `__repr__` — representação técnica

O `__repr__` define o que aparece quando você imprime um objeto no terminal ou no debugger. Sem ele, o Python mostra algo inútil como `<__main__.Funcionario object at 0x000001A2>`.

```
class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario

    def __repr__(self):
        return f"<Funcionario nome={self.nome}
salario={self.salario}>"

f = Funcionario("Carlos", 4500)
```

```
# Sem __repr__:
# print(f) → <__main__.Funcionario object at 0x000001A2> (inútil)

# Com __repr__:
print(f)      # <Funcionario nome=Carlos salario=4500>
print(repr(f)) # <Funcionario nome=Carlos salario=4500>

# Também aparece no terminal interativo:
>>> f
<Funcionario nome=Carlos salario=4500>
```

Você viu o `__repr__` nos modelos ORM do Módulo 3:

```
# app/models.py -- Módulo 3
class Departamento(Base):
    __tablename__ = "departamentos"
    id = Column(Integer, primary_key=True)
    nome = Column(String(100))

    def __repr__(self):
        return f"<Departamento id={self.id} nome={self.nome}>"

# No terminal:
>>> d = db.query(Departamento).first()
>>> print(d)
<Departamento id=1 nome=Tecnologia da Informação>
# Muito melhor que: <app.models.Departamento object at 0x0001>
```

5.3 `__str__` — representação legível

O `__str__` define o que aparece quando você usa `print()` ou `str()` em um objeto. É parecido com `__repr__`, mas voltado para o usuário final — não para o desenvolvedor.

```
class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario

    def __repr__(self):
        # Para o desenvolvedor: informação técnica
        return f"<Funcionario nome={self.nome} salario={self.salario}>"

    def __str__(self):
        # Para o usuário: texto amigável
        return f"{self.nome} – R$ {self.salario:.2f}"

f = Funcionario("Carlos Silva", 4500)
```

```
print(f)          # Carlos Silva - R$ 4500.00    (usa __str__)
print(repr(f))    # <Funcionario nome=Carlos ...> (usa __repr__)
print(str(f))     # Carlos Silva - R$ 4500.00    (usa __str__)
```

Método	Quando é chamado	Voltado para
<code>__repr__</code>	<code>repr(obj)</code> , terminal interativo, debugger	Desenvolvedor — informação técnica
<code>__str__</code>	<code>print(obj)</code> , <code>str(obj)</code> , f-strings	Usuário — texto amigável

💡 Qual implementar?

Se só criar o `__repr__`, ele vale para os dois casos.

Se criar os dois, o `__str__` tem prioridade no `print()`.

Nos modelos ORM do curso criamos só o `__repr__` -- é suficiente para debug.

5.4 Outros dunders que você vai encontrar

Dunder	O que define	Exemplo de uso
<code>__len__</code>	Tamanho do objeto com <code>len()</code>	<code>len(minha_lista_custom)</code>
<code>__eq__</code>	Comparação com <code>==</code>	<code>funcionario1 == funcionario2</code>
<code>__lt__</code>	Comparação com <code><</code> (menor que)	<code>sort()</code> em lista de objetos
<code>__add__</code>	Operador <code>+</code>	<code>vetor1 + vetor2</code>
<code>__getitem__</code>	Acesso com colchetes <code>obj[i]</code>	<code>minha_colecao[0]</code>
<code>__contains__</code>	Operador <code>in</code>	<code>"Carlos" in departamento</code>
<code>__enter__</code>	Início do bloco <code>with</code>	<code>with MeuContexto() as c:</code>
<code>__exit__</code>	Fim do bloco <code>with</code>	(fecha conexões, libera recursos)
<code>__call__</code>	Chamar o objeto como função	<code>meu_objeto()</code>
<code>__iter__</code>	Tornar o objeto iterável	<code>for item in meu_objeto:</code>

Não precisa decorar

Você não precisa decorar essa tabela.

O importante é saber que quando você vê `__algo__` em um código Python,

é um comportamento especial definido pela linguagem -- não um nome qualquer.

Com o tempo você vai reconhecer os mais comuns naturalmente.

6. Resumo — Tudo em um lugar só

6.1 Arquivos dunder

Arquivo	Onde fica	O que faz	Conteúdo típico
<code>__init__.py</code>	Dentro de cada pasta que é um pacote	Torna a pasta importável pelo Python	Vazio (na maioria dos casos)
<code>__main__.py</code>	Dentro de uma pasta/pacote	Permite executar a pasta com <code>python -m</code>	Código de inicialização do programa

6.2 Métodos dunder mais comuns

Método	Quando é chamado	O que você define
<code>__init__(self, ...)</code>	<code>MinhaClasse()</code>	Como inicializar o objeto
<code>__repr__(self)</code>	<code>repr(obj)</code> / terminal	Texto técnico do objeto
<code>__str__(self)</code>	<code>print(obj)</code> / <code>str(obj)</code>	Texto legível do objeto
<code>__len__(self)</code>	<code>len(obj)</code>	Tamanho do objeto
<code>__eq__(self, other)</code>	<code>obj1 == obj2</code>	Como comparar dois objetos

6.3 O `if __name__ == "__main__":`

```
# Padrão: separar o código reutilizável do código de teste/demo

def minha_funcao():
    pass

class MinhaClasse:
    pass

# Tudo abaixo SÓ roda quando o arquivo é executado diretamente
# NUNCA roda quando o arquivo é importado por outro
if __name__ == "__main__":
    obj = MinhaClasse()
    minha_funcao()
```

7. Exercícios de fixação

Exercício 1 — Verdadeiro ou Falso

Marque V (verdadeiro) ou F (falso):

- () O arquivo `__init__.py` precisa ter conteúdo para funcionar.
- () Toda pasta de um projeto Python precisa ter um `__init__.py`.
- () Sem o `__init__.py`, um import como `"from app.models import X"` vai falhar.
- () O `__init__` de uma classe é chamado automaticamente ao criar um objeto.
- () O `__repr__` define o texto que aparece ao usar `print()` em um objeto.
- () O bloco `if __name__ == "__main__":` nunca executa quando o arquivo é importado.
- () Você pode nomear suas funções como `__minhaFuncao__` sem problema.
- () O `__str__` e o `__repr__` fazem a mesma coisa.

Exercício 2 — Complete o código

Complete os espaços para que o código funcione corretamente:

```
class Produto:
    def ____(self, nome, preco):    # método construtor
        self.nome = ____          # atribuir o nome
        self.preco = preco
        self.ativo = ____          # valor padrão: True

    def ____(self):                 # representação técnica
        return f"<Produto nome={self.____} preco={self.preco}>"

    def __str__(self):              # representação legível
        return f"{self.nome} - R$ {self.____:.2f}"

# Testar
____ __name__ == "____":          # padrão de execução direta
    p = Produto("Notebook", 3500)
    print(p)                      # Notebook - R$ 3500.00
    print(repr(p))                # <Produto nome=Notebook preco=3500>
```

Exercício 3 — Diagnóstico de erro

O código abaixo gera um `ModuleNotFoundError`. O que está faltando e onde?

```
sistema/
├─ core/
```

```
|   ├── modelos.py  
|   └── banco.py  
└── rotas/  
    ├── clientes.py  
    └── main.py
```

```
# main.py  
from core.modelos import Cliente    # erro: ModuleNotFoundError  
from rotas.clientes import router  # erro: ModuleNotFoundError
```

Resposta: o que está faltando?

Como corrigir? Mostrar a estrutura correta:

Gabarito (Uso do Professor)

Exercício 1 — Verdadeiro ou Falso

- (F) O arquivo `__init__.py` precisa ter conteúdo para funcionar.
→ Pode ficar vazio. A presença é o que importa.
- (F) Toda pasta de um projeto Python precisa ter um `__init__.py`.
→ Só as pastas que você vai usar em imports.
- (V) Sem o `__init__.py`, um import como `"from app.models import X"` vai falhar.
→ `ModuleNotFoundError: No module named "app"`.
- (V) O `__init__` de uma classe é chamado automaticamente ao criar um objeto.
→ `Funcionario("Carlos", 4500)` chama `__init__` automaticamente.
- (F) O `__repr__` define o texto que aparece ao usar `print()` em um objeto.
→ O `__repr__` aparece no `repr()` e terminal. `print()` usa `__str__` se existir.
- (V) O bloco `if __name__ == "__main__":` nunca executa quando o arquivo é importado.
→ Ao importar, `__name__ == nome_do_arquivo`, não `"__main__"`.
- (F) Você pode nomear suas funções como `__minhaFuncao__` sem problema.
→ Nomes com duplo underscore são reservados. Evitar.
- (F) O `__str__` e o `__repr__` fazem a mesma coisa.
→ `__str__` é para o usuário (`print`). `__repr__` é para o desenvolvedor (`repr`).

Exercício 2 — Complete o código

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco
        self.ativo = True

    def __repr__(self):
        return f"<Produto nome={self.nome} preco={self.preco}>"

    def __str__(self):
```

```
        return f"{self.nome} - R$ {self.preco:.2f}"

if __name__ == "__main__":
    p = Produto("Notebook", 3500)
    print(p)
    print(repr(p))
```

Exercício 3 — Diagnóstico de erro

```
# O que está faltando:
# __init__.py dentro de core/ e de rotas/

# Estrutura correta:
sistema/
├── core/
│   ├── __init__.py  ← FALTAVA
│   ├── modelos.py
│   └── banco.py
├── rotas/
│   ├── __init__.py  ← FALTAVA
│   └── clientes.py
└── main.py

# Após criar os dois arquivos vazios:
# from core.modelos import Cliente  → funciona
# from rotas.clientes import router → funciona
```